

# MANUAL TÉCNICO

## 1. Introducción

El presente documento describe el diseño del analizador léxico del proyecto MedLexer, el cual tiene como objetivo interpretar archivos con extensión .med escritos en el lenguaje MedLang, orientado a la gestión hospitalaria.

El sistema permite reconocer estructuras como pacientes, médicos, citas y diagnósticos, detectando errores léxicos con precisión de línea y columna. Para ello, se implementa un Autómata Finito Determinista (AFD) de forma manual en C++.

## 2. Definición del Lenguaje MedLang

### 2.1 Lista de Tokens

Los tokens reconocidos por el lenguaje son los siguientes:

#### Palabras reservadas

- HOSPITAL
- PACIENTES
- MEDICOS
- CITAS
- DIAGNOSTICOS
- paciente
- medico
- cita
- diagnostico
- con

#### Identificadores (atributos)

- edad
- tipo\_sangre
- habitacion
- especialidad
- codigo
- fecha
- hora
- condicion
- medicamento

- dosis

### **Literales**

- Cadena: "Texto"
- Entero: 45
- Fecha: 2025-04-10
- Hora: 09:30
- Código: MED-001

### **Enumeraciones**

#### **Especialidades:**

- CARDIOLOGIA
- NEUROLOGIA
- PEDIATRIA
- CIRUGIA
- MEDICINA\_GENERAL
- ONCOLOGIA

#### **Frecuencia de dosis:**

- DIARIA
- CADA\_8\_HORAS
- CADA\_12\_HORAS
- SEMANAL

### **Tipos de sangre**

- A+, A-, B+, B-, O+, O-, AB+, AB-

### **Símbolos**

- {}[]:.,;

### **Tokens especiales**

- ERROR
- EOF

## **2.2 Reglas Léxicas**

Las reglas para la formación de tokens son:

- Identificadores: comienzan con letra y pueden contener letras y números.

- Cadenas: texto entre comillas dobles.
- Enteros: secuencia de dígitos.
- Fecha: formato AAAA-MM-DD.
- Hora: formato HH:MM en 24 horas.
- Código: tres letras mayúsculas, guion y números.

Validaciones:

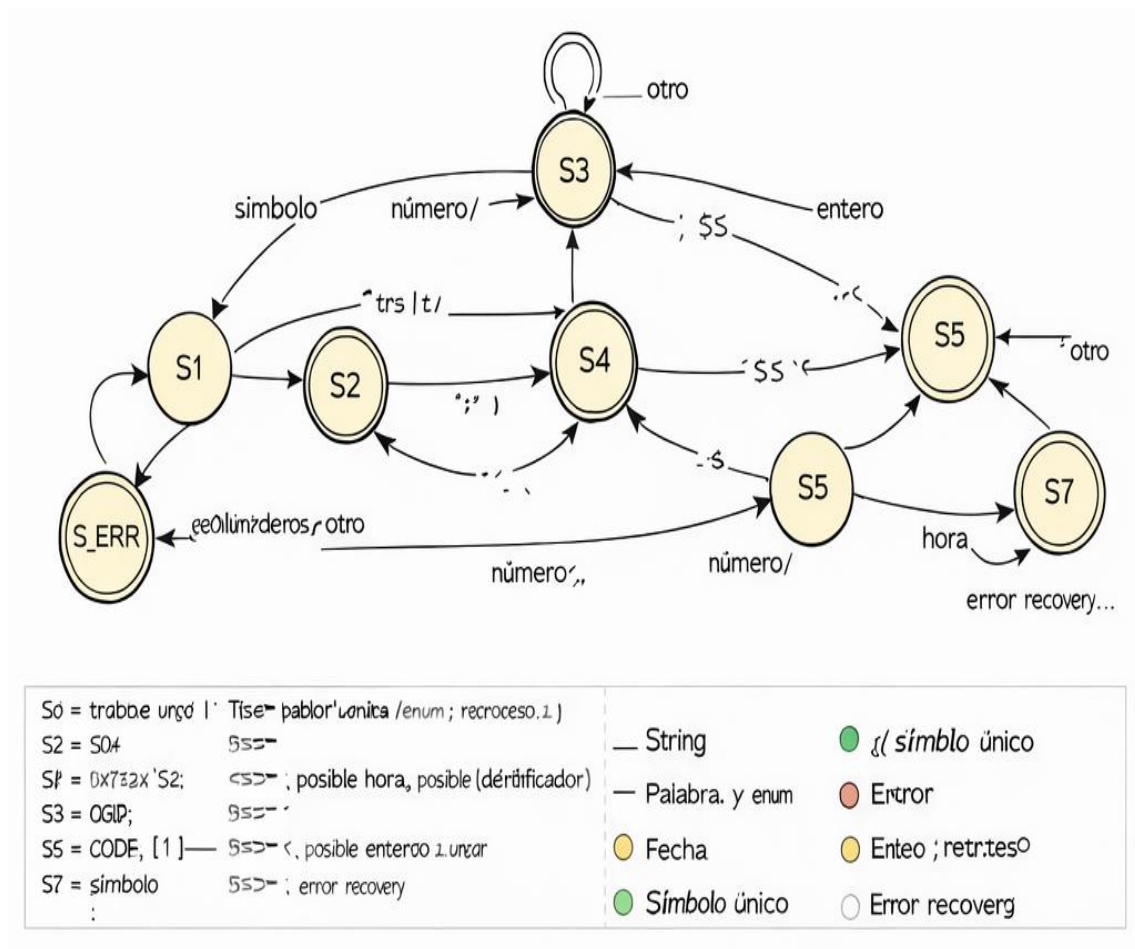
- Fecha: mes (01–12), día (01–31)
- Hora: 00:00 a 23:59
- Tipo de sangre válido

### 3. Diseño del AFD

#### 3.1 Descripción

El AFD permite reconocer tokens leyendo carácter por carácter y cambiando de estado según el tipo de entrada. Cada estado representa un tipo de token en construcción.

#### 3.2 Diagrama del AFD



### 3.3 Tabla de Transiciones

| Estado | Entrada      | Nuestro next    | Acciones / Notas                 |
|--------|--------------|-----------------|----------------------------------|
| S0     | ws           | S0              | ignora                           |
| S0     | "            | S2              | start string                     |
| S0     | [A-Z]        | S1              | posible keyword/código           |
| S0     | [a-z_]       | S1              | ident                            |
| S0     | [0-9]        | S3              | number                           |
| S0     | { } [ ] : ,  | SF_DELIM        | token símbolo                    |
| S0     | else         | S8              | error                            |
| S1     | [A-Za-z0-9_] | S1              | sigue token                      |
| S1     | -            | S6              | código u identificador especial  |
| S1     | resto        | FIN y retroceso | evalúa palabra reservada/enum/ID |
| S2     | \            | S2              | escape                           |
| S2     | "            | FIN             | string válida                    |
| S2     | EOF, \n      | S8              | error cadena no cerrada          |
| S3     | [0-9]        | S3              | sigue entero                     |
| S3     | :            | S5              | posible hora                     |
| S3     | -            | S4              | posible fecha                    |
| S3     | otras        | FIN retroceso   | entero válido                    |
| S4     | [0-9]        | S4              | fecha parcial (validar formatos) |
| S4     | otras        | FIN u error     | validar AAAA-MM-DD               |
| S5     | [0-9]        | S5              | hora parcial                     |
| S5     | otras        | FIN u error     | validar HH:MM                    |
| S6     | [0-9]        | S6              | código continuación              |
| S6     | otras        | FIN             | evaluar tipo código o id         |
| S8     | cualquier    | S0 (reset)      | acumula error, busca separador   |

### 3.4 Aceptación

- S1 → TOKEN\_IDENT / TOKEN\_KEYWORD / TOKEN\_ENUM / TOKEN\_ID / TOKEN\_CODIGO
- S2 → TOKEN\_STRING

- S3 → TOKEN\_INT
- S4 → TOKEN\_DATE
- S5 → TOKEN\_TIME
- S7 → TOKEN\_SYMBOL
- S8 → TOKEN\_ERROR

### 3.5 Estados y significado

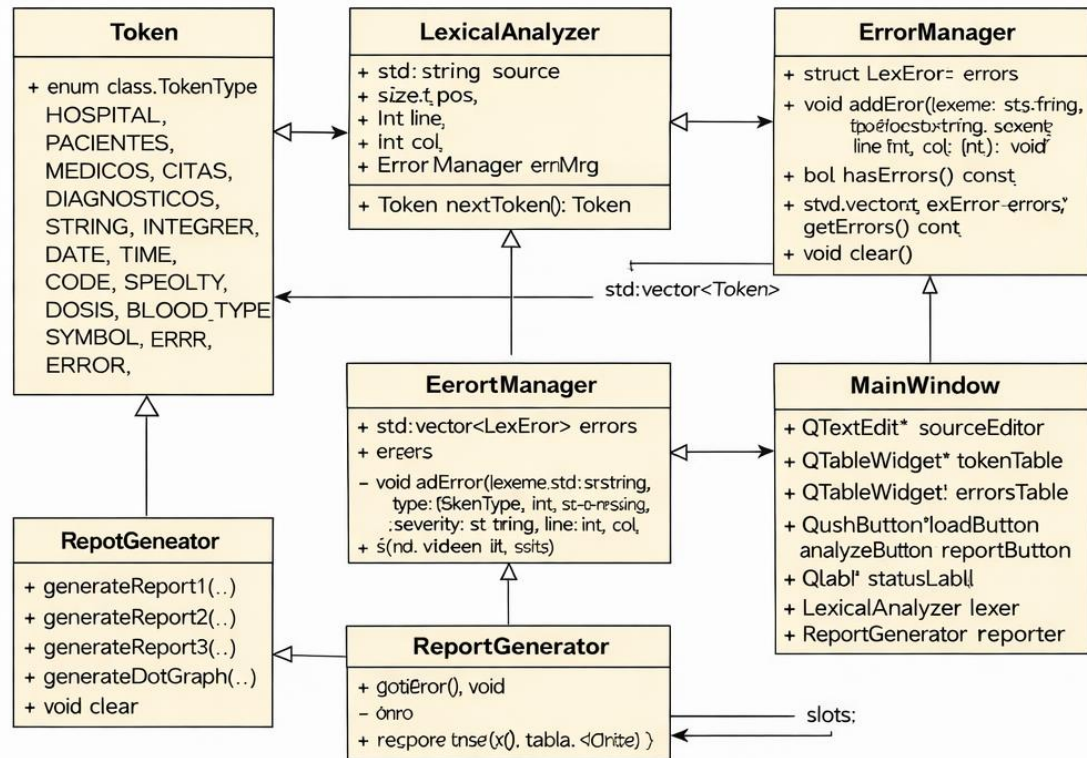
- S0 = inicio
- S1 = reconocimiento de palabra ([A-Za-z\_])
- S2 = cadenas ("")
- S3 = números enteros ([0-9])
- S4 = fechas parcial (aaaa-)
- S5 = horas parcial (hh:)
- S6 = códigos con guión (X)
- S7 = símbolo único ({ } [ ] : ,)
- S8 = estado de error
- SF = aceptación final de cada token tipo

### Transiciones (principal)

- S0:
  - " → S2
  - [A-Z] → S1 (posible keyword / enumerado / código)
  - [a-z\_] → S1 (identificador)
  - [0-9] → S3
  - { } [ ] : , → SF\_DELIM
  - - / . / otros → S8 (error)
  - WS → S0 (skip)
- S1:
  - [A-Za-z0-9\_] → S1 continuo
  - - → S6 si patrón es código
  - otro → aceptación token (palabra/reservada/enum/identificador), retroceso

- S2:
  - " → aceptación string
  - \ → escape (opcional) → sigue en S2
  - \n o EOF → error cadena sin cerrar
  - otro → S2
- S3:
  - [0-9] → S3
  - : → S5 (posible hora)
  - - → S4 (posible fecha)
  - otro → aceptación entero, retroceso
- S4 (fecha tras YYYY-):
  - [0-1][0-9] etc. → validar longitud y rangos
  - tras dos grupos y guiones → aceptación fecha
- S5 (hora tras HH:):
  - [0-5][0-9] → aceptación hora
- S6 (código reevaluar):
  - [0-9]+ → estado código
  - otro → token posible palabra/reservado / error
- S8:
  - consume hasta separador (error recuperable)

#### 4. Diseño del Sistema (UML)



## 5. Casos de Prueba

| No | Caso de Prueba          | Entrada         | Resultado esperado |
|----|-------------------------|-----------------|--------------------|
| 1  | Archivo Válido          | Archivo válido  | Sin errores        |
| 2  | Cadena sin cerrar       | "Jimmy Hurtarte | Error              |
| 3  | Fecha Inválida          | 2025-13-01      | Error fecha        |
| 4  | Hora Inválida           | 25:00           | Error hora         |
| 5  | Tipo de Sangre Inválido | "O++"           | Error tipo sangre  |
| 6  | Especialidad Inválida   | CARDIO          | Error especialidad |
| 7  | Código Inválido         | MED001          | Error código       |
| 8  | Símbolo inesperado      | @               | Error símbolo      |
| 9  | Número mal formado      | 4a5             | Error número       |
| 10 | Hora incompleta         | 09:             | Error hora         |

| No | Caso de Prueba    | Entrada                        | Resultado esperado |
|----|-------------------|--------------------------------|--------------------|
| 11 | Fecha incompleta  | 2025-04                        | Error fecha        |
| 12 | Múltiples errores | hora: 30:90, tipo_sangre: "X+" | Detectar todos     |
| 13 | Falta de comillas | Jimmy Hurtarte                 | Error cadena       |
| 14 | Archivo vacío     | (empty)                        | Sin tokens         |
| 15 | Código sin guion  | mezcla válida/errónea          | Detectar ambos     |

### Descripción general

El sistema desarrollado es un Sistema de Gestión Hospitalaria, implementado en C++, cuyo objetivo es administrar información de pacientes, médicos, citas y diagnósticos, permitiendo además la generación de reportes en HTML y visualizaciones mediante Graphviz.

El sistema está estructurado bajo un enfoque modular y orientado a objetos.

### REQUERIMIENTOS TÉCNICOS

#### Hardware

Para el correcto funcionamiento del sistema, se requieren las siguientes especificaciones mínimas:

- **Procesador:** Intel Core i3 o superior
- **Memoria RAM:** 4 GB mínimo (8 GB recomendado)
- **Disco Duro:** 500 MB libres (para ejecutables, reportes y grafos)
- **Resolución de pantalla:** 1366x768 o superior

#### Software

- **Sistema Operativo:**
  - Windows 10 o superior
  - Linux (Ubuntu 20.04+)
  - macOS
- **Compilador:**
  - g++ (MinGW / GCC)
- **Herramientas:**
  - Graphviz (para generación de grafos)
  - Navegador web (Chrome, Edge, Firefox)



- **Librerías utilizadas:**

- <vector>
- <map>
- <algorithm>
- <fstream>

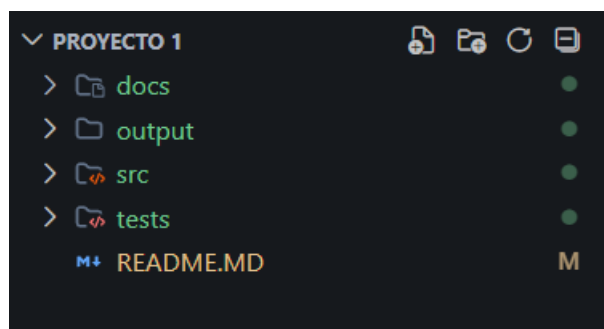
- **Dependencias:**

- No requiere frameworks externos
- No utiliza base de datos (manejo en memoria)

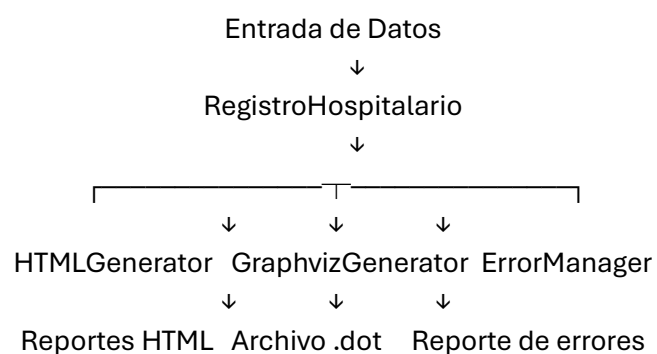
## DESCRIPCIÓN TÉCNICA DEL SISTEMA

### Arquitectura

El sistema utiliza una arquitectura modular basada en Programación Orientada a Objetos.



### Diagrama de Bloques



### Diagrama de Secuencia

Usuario → Sistema → RegistroHospitalario  
→ Validación (ErrorManager)  
→ Generación de reportes (HTMLGenerator)  
→ Generación de grafo (GraphvizGenerator)

## **Diagrama de Casos de Uso**

Actor: Usuario

- Registrar pacientes
- Registrar médicos
- Registrar citas
- Generar reportes
- Visualizar grafo
- Consultar estadísticas

## **Diccionario de Datos**

Aunque no se usa base de datos, las estructuras funcionan como tablas:

### **Paciente**

- nombre (string)
- edad (int)
- tipoSangre (string)
- habitacion (int)

### **Medico**

- nombre (string)
- especialidad (string)
- codigo (string)

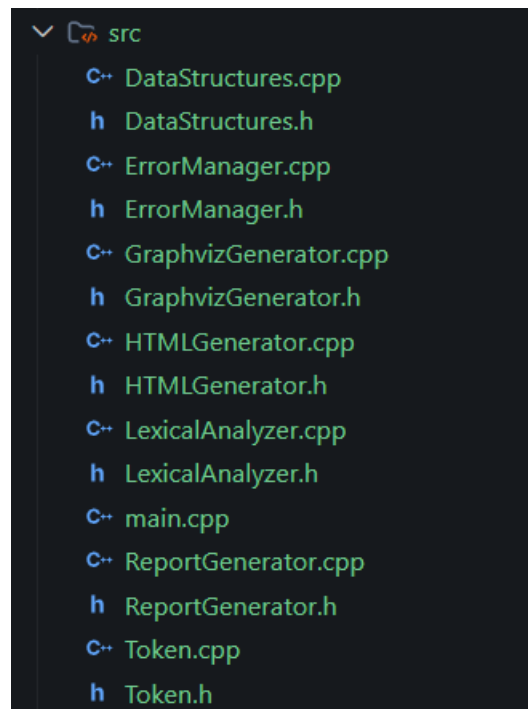
### **Cita**

- paciente (string)
- medico (string)
- fecha (string)
- hora (string)
- conflicto (bool)

### **Diagnostico**

- paciente (string)
- condicion (string)
- medicamento (string)
- dosis (string)

## Estructura de Archivos



## INSTALACIÓN Y CONFIGURACIÓN

### Pasos de Instalación

1. Instalar compilador g++
2. Instalar Graphviz
3. Clonar o descargar el proyecto
4. Compilar:

```
g++ src/*.cpp -o sistema
```

5. Ejecutar:

```
./sistema
```

### Configuración

- No requiere archivo .env
- No requiere base de datos
- Solo verificar:
  - Ruta de salida de archivos .html y .dot

## DOCUMENTACIÓN DEL CÓDIGO

### Descripción de Módulos

- **DataStructures:** Manejo de datos
- **ErrorManager:** Control de errores

- **HTMLGenerator:** Generación de reportes
- **GraphvizGenerator:** Visualización del sistema

### **Funciones importantes**

#### **buscarPaciente(nombre)**

- Entrada: nombre (string)
- Salida: puntero a Paciente
- Función: busca un paciente

#### **edadPromedio()**

- Entrada: ninguna
- Salida: double
- Función: calcula edad promedio

#### **generarGrafo()**

- Entrada: nombre archivo
- Salida: bool
- Función: genera archivo .dot

#### **generarReportePacientes()**

- Entrada: nombre archivo
- Salida: bool
- Función: genera HTML

## **MANTENIMIENTO Y SEGURIDAD**

### **Plan de Mantenimiento**

- Validar datos de entrada
- Limpiar registros con clear()
- Revisar conflictos de citas
- Actualizar reportes periódicamente

## **SOLUCIÓN DE PROBLEMAS**

### **Errores Comunes**

#### **No genera archivo .dot**

✓ Solución:

- Verificar permisos de escritura
- 

#### **No genera imagen**

✓ Solución:

`dot -Tpng archivo.dot -o archivo.png`

---

#### **HTML no abre**

✓ Solución:

- Abrir con navegador moderno
- 

#### **Error de compilación**

✓ Solución:

`g++ src/*.cpp -o sistema`

---

#### **Log de errores**

El sistema utiliza:

ErrorManager

Permite:

- Ver errores con `printErrors()`
- Identificar línea y columna

Diagrama en GRAPHVIZ

